Sequenced Collections and Maps 2



Chapter Topics

- How collections, sets, and maps with a *well-defined encounter order* have been retrofitted as *sequenced* types in the `java.util` package using the `SequencedCollection<E>`, the `SequencedSet<E>`, and the `SequencedMap<K, V>` interfaces that allow *uniform operations on elements at both ends* and provide a *reverse-ordered view*.
- How to create *unmodifiable sequenced* collections, sets, and maps.
- How concurrent collections and maps in the `java.util.concurrent` package have been retrofitted as *sequenced concurrent* types.



Prerequisites

- How to create and use concrete classes of the `Collection`, the `List`, the `Set`, the `Deque`, and the `Map` interfaces in the `java.util` package that support operations to add, remove, update, retrieve and sort elements.
- How to create unmodifiable collections, sets, and maps using methods in the `java.util.Collections` utility class.
- Familiarity with the concurrent collections and maps in the `java.util.concurrent` package.

Java SE 21 Developer Professional Exam Objectives	
[5.1] Create arrays, List, Set, Map and Deque collections, and add, remove, update, retrieve and sort their elements.	*This chapter only covers sequenced collections, lists, set, deque, and maps.*

As of Java 21, new interfaces for *sequenced* types that exhibit an inherent *well-defined encounter order* have been integrated into the Java Collections Framework. These interfaces are seamlessly retrofitted into the existing type hierarchy to provide a uniform set of operations at both ends, including a reverse-ordered view. This has been achieved without deprecation of any existing code in the Java Collections Framework. Introduction of these sequenced interfaces has resulted in the framework becoming more versatile and powerful than before.

We assume familiarity with the Collections Framework prior to Java 21, especially the core interfaces in the `java.util` package.

2.1 Collections with Well-Defined Encounter Order

A common property of many collections in the Java Collections Framework is that they have an inherent *well-defined encounter order*—meaning that they have a well-defined first element, second element, and so on, right to the last element. This order imposes a relationship between any two elements in the collection, where either one element is before or after the other element. A consequence of this relationship is that the one before is then closer to the first element and the one after is closer to the last element. An important implication of the defined encounter order is that, not only can the elements be processed from the first element to the last element, but there is a reverse encounter order that allows elements to be processed from the last element to the first element.

Note that the defined encounter order of a collection does not imply how the elements are physically stored in memory.

In the past, there has not been a universal collection supertype in the Java Collections Framework to represent a sequenced collection—that is, a sequence of elements that has a well-defined encounter order allowing uniform operations at both ends and offering a reverse-ordered view. The `List` and `SortedSet` interfaces inherently define an encounter order, but their common supertype `Collection` is too general and does not define an encounter order. Similarly the `Set` interface does not define an encounter order, but its subtypes `SortedSet` and `LinkedHashSet` do.

There was also either no support or inconsistent support for a uniform set of operations across collections that exhibit a defined encounter order. The first element of a list can be accessed by the `get(0)` method call, but for a sorted set, we need to call the `first()` method. Moreover, a deque provides its own `getFirst()` method for the same purpose. Accessing the last element is also not consistent: either different methods in each kind of collection (for example: `list.get(list.size-1)`, `sorted-Set.last()`, `deque.getLast()`), or no such operation at all (as in the case of a `LinkedHashSet`).

Although all collections provided the `iterator()` method to obtain an iterator to process elements from the first to the last element in a collection, there was no standard way to iterate in the reverse direction. One could use a reverse iterator obtained by calling a specific method in the collection (for example: `list.listIterator()` that allows traversal in either direction, `navigableSet.descendingIterator()`, `deque.descendingIterator()`), if such an iterator existed (for example, there is no reverse iterator in the case of a `LinkedHashSet`).

The uniform support for sequenced collections, sets, and maps enhances the Java Collections Framework, without affecting how the framework was used previously.

2.2 Sequenced Collections

The generic `Collection<E>` interface is a generalized interface for maintaining collections, and is the root of the inheritance hierarchy for collections, as shown in **Figure 2.1**. It shows the inheritance relationship between the core collection interfaces and the corresponding implementations in the `java.util` package. None of the concrete implementations inherit directly from the `Collection` interface. The abstract classes that provide the basis on which concrete classes are implemented are not shown in **Figure 2.1**.

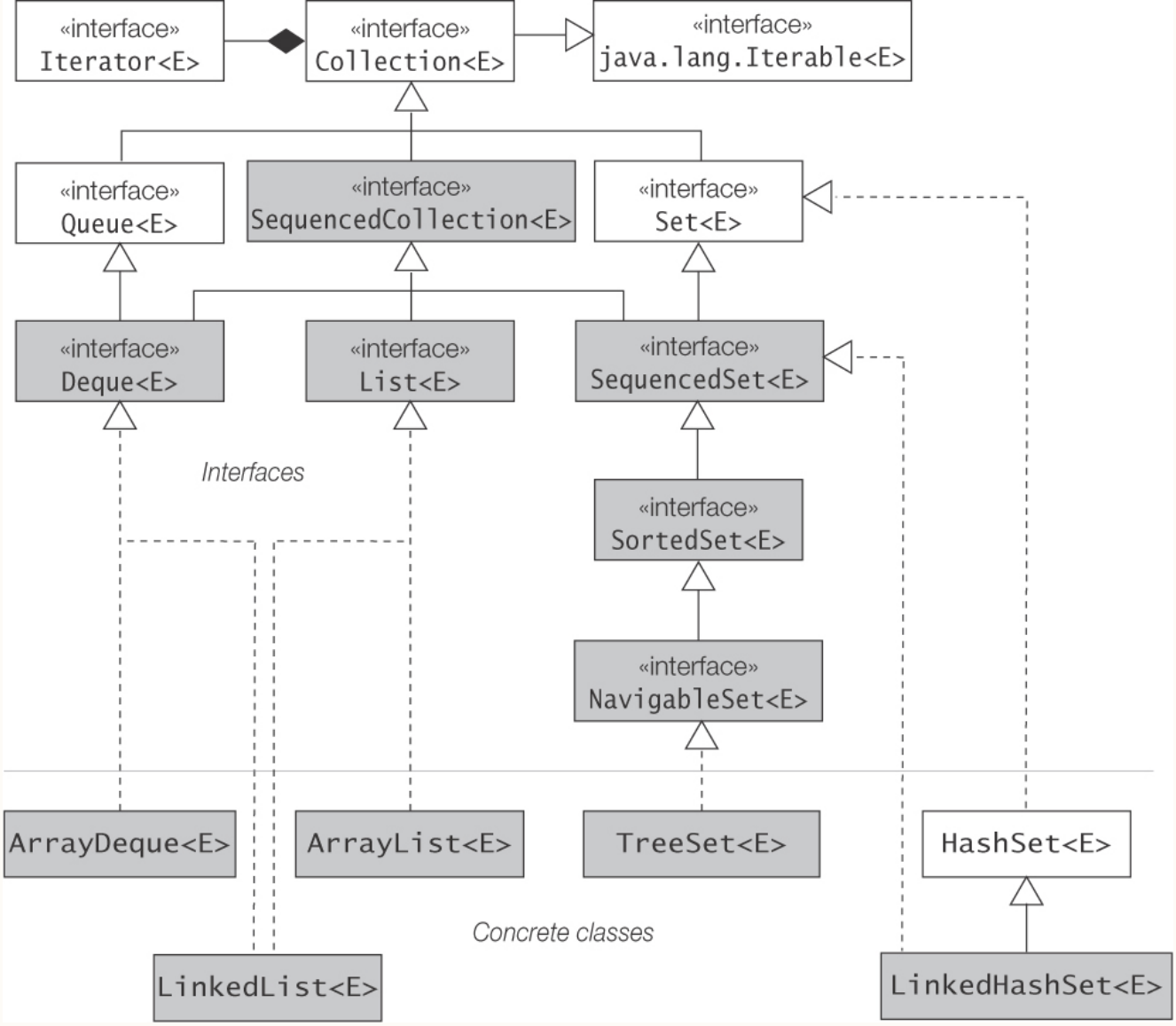


Figure 2.1 *Core Collections Inheritance Hierarchy*

Figure 2.1 also shows how the `SequencedCollection` and the `SequencedSet` interfaces that provide support for *sequenced collections* are retrofitted onto the inheritance hierarchy of collections. These sequenced interfaces and their sequenced subtypes (shown colored grey in Figure 2.1) impose an encounter order on their elements and define operations on elements at either end of a sequenced collection. However, the interfaces `Collection`, `Set`, and `Queue` do not represent sequenced collections, since their collections do not enforce an encounter order on their elements.

The core collection interfaces and their concrete implementations are summarized in Table 2.1. Rows colored light grey in Table 2.1 represent sequenced interfaces.

Table 2.1 *Summary of Core Collection Interfaces*

Interface	Description	Select Concrete Classes
`Collection<E>`	A basic interface that defines the normal operations that allow a collection of objects to be maintained or handled as a single unit.	`ArrayList<E>`, `LinkedList<E>`, `HashSet<E>`, `LinkedHashSet<E>`, `TreeSet<E>`,

Interface	Description	Select Concrete Classes
		ArrayDeque<E>, PriorityQueue<E>
SequencedCollection<E> extends Collection<E>	A sequenced collection maintains its elements in a well-defined encounter order, allows operations at both ends, and that is reversible.	ArrayList<E>, LinkedList<E>, LinkedHashSet<E>, TreeSet<E>, ArrayDeque<E>
List<E> extends SequencedCollection<E>	A list maintains an ordered collection that allows duplicates, where the order of the elements is *positional*, and individual elements can be accessed randomly according to their position in the list.	ArrayList<E>, LinkedList<E>
Set<E> extends Collection<E>	A set represents its mathematical namesake: a *set* of unique elements.	HashSet<E>, LinkedHashSet<E>, TreeSet<E>
SequencedSet<E> extends SequencedCollection<E>, Set<E>	A sequenced set models both a set that has a well-defined encounter order and a sequenced collection with unique elements.	LinkedHashSet<E>, TreeSet<E>
SortedSet<E> extends SequencedSet<E>	A sorted set provides a total ordering on its elements, required to maintain the elements in some sort order.	TreeSet<E>

Interface	Description	Select Concrete Classes
NavigableSet<E> extends SortedSet<E>	A navigable set is a sorted set extended with navigable methods to locate the closest match to a given element.	TreeSet<E>
Queue<E> extends Collection<E>	A queue maintains elements that need to be processed according to some policy—that is, insertion at one end and removal at the other, usually as FIFO (first in, first out).	PriorityQueue<E>
Deque<E> extends SequencedCollection<E>, Queue<E>	A deque maintains a queue whose elements can be processed at both ends: *d*ouble-ended *que*ue.	ArrayDeque<E>, LinkedList<E>

The `SequencedCollection<E>` Interface

A sequenced collection is a collection whose elements exhibit a defined encounter order. The behavior of a sequenced collection is dictated by the `SequencedCollection` interface that provides `default` methods to *add*, *retrieve*, and *remove* elements at *both ends*, including a method to create a *reverse-ordered view* of such a collection.

Methods to add, retrieve, and remove elements at both ends are *promoted* from the `Deque` interface that already defined these methods before the introduction of sequenced collections. Pertinent API details of the methods in the `SequencedCollection` interface are given below.

java.util.SequencedCollection<E>

Click here to view code image

```
// Optional add methods (promoted from the Deque<E> interface):
default void addFirst(E element)
default void addLast(E element)
```

Add the specified element to the sequenced collection as the first or the last element, respectively. If the element is added, it becomes the first or the last element in the collection, respectively.

The methods will throw a `NullPointerException` if the specified element is `null` and an `UnsupportedOperationException` if the operation is not supported.

The default implementations of these methods in the `SequencedCollection` interface throws an `UnsupportedOperationException`.

Click here to view code image

```
// Get methods (promoted from the Deque<E> interface):
default E getFirst()
default E getLast()
```

Get the first or the last element from the collection, respectively, but do not remove it.

The methods throw a `NoSuchElementException` if the collection is empty.

The default implementations of these methods in the `SequencedCollection<E>` interface use an iterator to get the appropriate element.

Click here to view code image

```
// Optional remove methods (promoted from the Deque<E> interface):
default E removeFirst()
default E removeLast()
```

Remove and return the first or the last element from the sequenced collection, respectively.

The methods throw a `NoSuchElementException` if the collection is empty and an `UnsupportedOperationException` if the operation is not supported.

The default implementations of these methods in the `SequencedCollection` interface use an iterator to remove the appropriate element.

Click here to view code image

```
// Create the reverse-ordered view:
SequencedCollection<E> reversed()
```

Returns a reverse-ordered view of this collection. Modification to the reverse-ordered view may or may not be written through to the underlying collection. Any changes to the underlying collection may or may not be visible in the reverse-ordered view.

The `SequencedCollection<E>` interface does not provide an implementation for this method, and is thus declared as an `abstract` method.

Note that the `SequencedCollection` interface provides `default` implementations for the add, retrieve, and remove methods. The `get*()` and `remove*()` methods can throw a `NoSuchElementException`, if the collection is empty (`*` in the method name can be the suffix `First` or `Last`). The `add*()` and `remove*()` methods are *optional*, meaning that a collection can choose whether to provide such an operation or not. If an optional method is not supported, it should throw an `UnsupportedOperationException` in its implementation.

Consider a `List` which has *insertion order* as its defined encounter order. Adding, retrieving, and removing elements at either end of a list does not structurally affect the defined encounter order. The list still remains in insertion order. However, a `SortedSet` has a *sort order* as its defined encounter order that determines by comparison where an element is placed in the sorted set. Allowing to add an element at either end of a sorted set can invalidate the sort order, as this might not be the right place for the element being added according to the sort order.

An *unmodifiable sequenced collection* does not allow `add*()` and `remove*()` methods, as that would violate its integrity (**p. 62**). These optional methods are thus not allowed on unmodifiable sequenced collections. Thus, the optional methods of a sequenced collection ensure that the integrity of its defined encounter order is not compromised.

The *reverse-ordered view* of the sequenced collection created by the abstract `reversed()` method is a sequenced collection. Whether changes in the view are visible in the underlying sequenced collection and vice versa, is dependent on the implementation of the method provided by the sequenced collection. Given the reverse-ordered view, it is then possible to process the elements of a sequenced collection in both directions using any of the usual iteration mechanisms (for example, an enhanced `for(:)` loop, an explicit iterator, the `forEach()` method, or iterating using a stream or an array).

Table 2.2 shows the behavior of sequenced methods in select *concrete sequenced collections*, as defined by the `SequencedCollection` interface:

covariant override	The collection overrides the method from its supertype, but with a covariant return.
inherits	The collection inherits the method from its supertype.
overrides	The collection overrides the method from its supertype.
throws UnsupportedOperationException	The method throws an `UnsupportedOperationException`, as the collection does not support the operation.
may throw `NoSuchElementException`	The method throws a `NoSuchElementException`, if the collection is empty.
may throw NullPointerException	The method throws a `NullPointerException`, if the element to be added is `null`.

returns ...	The method returns an object of the specified reference type.

From **Table 2.2**, we see that the `add*()` methods of the concrete class `TreeSet` will always throw an `UnsupportedOperationException` so that its defined encounter order (based on sorted elements) is not compromised. Only the `ArrayDeque` class throws a `NullPointerException` if an attempt is made to add the `null` value.

Whether inherited or overridden, the `get*()` and the `remove*()` methods will throw a `NoSuchElementException` if the collection is empty.

The `reversed()` method of the concrete sequenced collections returns a reverse-ordered view that is of the supertype, except in the case of a `LinkedList`.

Table 2.2 *Sequenced Methods in Select Concrete Sequenced Collections*

Concrete Collections	**`addFirst()` `addLast()` (Optional)**	**`getFirst()` `getLast()`**	**`removeFirst()` `removeLast()`** (Optional)	**`reversed()`**
`ArrayList<E>` implements `List<E>`	*overrides*	*overrides, may throw* `NoSuchElement-Exception`	*overrides, may throw* `NoSuchElement-Exception`	*inherits, returns* `List<E>`
`TreeSet<E>` implements `NavigableSet<E>`	*overrides, throws* `Unsupported-Operation-Exception`	*inherits, may throw* `NoSuchElement-Exception`	*inherits, may throw* `NoSuchElement-Exception`	*inherits, returns* `NavigableSet<E>`
`LinkedHashSet<E>` implements `SequencedSet<E>`	*overrides*	*overrides, may throw* `NoSuchElement-Exception`	*overrides, may throw* `NoSuchElement-Exception`	*overrides, returns* `SequencedSet<E>`
`ArrayDeque<E>` implements `Deque<E>`	*overrides, may throw* `NullPointer-Exception`	*overrides, may throw* `NoSuchElement-Exception`	*overrides, may throw* `NoSuchElement-Exception`	*inherits, returns* `Deque<E>`

Concrete Collections	`addFirst()` `addLast()` **(Optional)**	`getFirst()` `getLast()`	`removeFirst()` `removeLast()` (Optional)	`reversed()`
`LinkedList<E>` implements `List<E>`, `Deque<E>`	*overrides*	*overrides, may throw* `NoSuchElement-Exception`	*overrides, may throw* `NoSuchElement-Exception`	*covariant override* `LinkedList<E>`

2.3 Sequenced Lists

The `List<E>` Interface

The `List` interface provides default implementations of all methods in its super-interface `SequencedCollection`, and covariantly overriding the `reversed()` method to return a `List`.

java.util.List<E>

Click here to view code image

```
// Create a reverse-ordered view:
List<E> reversed()
```

Returns a reverse-ordered view of this list.

This method is a covariant override of the method from the `SequencedCollection` interface, implemented as a default method, returning a `List` instead of a `SequencedCollection`.

The `ArrayList<E>` Class

The `ArrayList` class overrides the default implementations of the sequence methods in the `List` interface. These sequence methods make the `ArrayList` all the more versatile to process elements at both ends.

Elements can be processed conveniently in either direction:

Click here to view code image

```
jshell> List<Integer> intList = new ArrayList<>(List.of(4, 5, 2, 3));
intList ==> [4, 5, 2, 3]
jshell> intList.stream().mapToInt(Integer::intValue).sum();
$3 ==> 14
jshell> List<Integer> revIntList = intList.reversed();
```

```
revIntList ==> [3, 2, 5, 4]
jshell> revIntList.stream().mapToInt(Integer::intValue).sum();
$5 ==> 14
```

Changes in either the reserve-ordered view or the underlying array list are written through:

Click here to view code image

```
jshell> revIntList.removeLast();
$6 ==> 4
jshell> revIntList
revIntList ==> [3, 2, 5]
jshell> intList
intList ==> [5, 2, 3]

jshell> intList.removeFirst();
$9 ==> 5
jshell> intList
intList ==> [2, 3]
jshell> revIntList
revIntList ==> [3, 2]
```

2.4 Sequenced Sets

The `SequencedSet<E>` Interface

The `SequencedSet` interface inherits the behavior of both `Set` and `SequencedCollection` interfaces, as such a sequenced set can be used to model both a *set* that has a defined encounter order or a *sequenced collection* with no duplicates.

The `SequencedSet` interface inherits the `add*()`, `get*()`, and the `remove*()` methods for operations at both ends from its superinterface `SequencedCollection`, and only covariantly overrides the `reversed()` method to return a `SequencedSet`.

java.util.SequencedSet<E>

Click here to view code image

```
// Create a reverse-ordered view:
SequencedSet<E> reversed()
```

Returns a reverse-ordered view of this set.

This method is a covariant override of the method from `SequencedCollection` interface, defined as an `abstract` method, returning a `SequencedSet` instead of a `SequencedCollection`.

The `SortedSet<E>` Interface

The `SortedSet` interface provides default implementations of the `add*()`, `get*()`, and `remove*()` methods for operations at both ends by overriding these methods from its superinterface `SequencedSet`, and also covariantly overriding the `reversed()` method to return a `SortedSet`.

The `add*()` methods throw an `UnsupportedOperationException`, as *explicit positioning* of elements is not allowed in a sorted set since elements are positioned by relative comparison to preserve the sort order. Elements should therefore be added by methods `add()` and `addAll()` that uphold the sort order.

The implementations of the `get*()` and `remove*()` methods call the `first()` and `remove()` methods defined by the `SortedSet` interface.

The `NavigableSet<E>` Interface

The `NavigableSet` interface inherits the `add*()` and the `get*()` methods from its superinterface `SortedSet`, but provides default implementations of the `remove*()` and the `reversed()` methods, the latter implemented to covariantly return a `NavigableSet`.

The implementations of the `remove*()` methods call the `poll*()` methods defined by the `NavigableSet` interface.

The `TreeSet<E>` Class

The `TreeSet` class inherits all sequence methods from its superinterface `NavigableSet`, except the `add*()` methods that are overridden to throw an `UnsupportedOperationException`, as *explicit positioning* of elements is not allowed in a `TreeSet` since elements have some sort order. The `jshell` session below illustrates that the `add*()` methods are not supported by the `TreeSet` class:

Click here to view code image

```
jshell> var sortedSet = new TreeSet<String>();
sortedSet ==> []
jshell> sortedSet.add("Where"); sortedSet.add("What");
jshell> sortedSet
sortedSet ==> [What, Where]
jshell> sortedSet.addFirst("Who");
|  Exception java.lang.UnsupportedOperationException
|        at TreeSet.addFirst (TreeSet.java:476)
|        at (#16:1)
```

The `LinkedHashSet<E>` Class

The `LinkedHashSet` class overrides all sequence methods from its superinterfaces `List` and `SequencedSet`. The `add*()` methods in a `LinkedHashSet` handle the element to be added in a special way. The element is added at the appropriate end, unless it is a duplicate, in which case, the existing element is repositioned at the appropriate end. The `jshell` session below illustrates how a duplicate in a `LinkedHashSet` will be moved to the appropriate end of the set:

Click here to view code image

```
jshell> var lhs = new LinkedHashSet<>();
lhs ==> []
jshell> lhs.addFirst("Breakfast"); lhs.addFirst("Lunch"); lhs.addFirst("Dinner");
jshell> lhs
lhs ==> [Dinner, Lunch, Breakfast]
jshell> lhs.addFirst("Lunch");                    // Add duplicate.
jshell> lhs
lhs ==> [Lunch, Dinner, Breakfast]
```

2.5 Deques

The `add*()`, `get*()`, and `remove*()` methods are *promoted* from the `Deque` interface to the `SequencedCollection` interface. A sequenced collection is therefore a generalization of a deque, with the added support for a reverse-ordered view. Its `reversed()` method is a covariant override of the method from the `SequencedCollection` interface, implemented as a default method, returning a `Deque` instead of a `SequencedCollection`.

The `ArrayDeque<E>` Class

The `ArrayDeque` class overrides `add*()`, `get*()`, and the `remove*()` methods from the `Deque` interface, but inherits the `reversed()` method. Changes to the reverse-ordered view or to the underlying array deque are *written through*:

Click here to view code image

```
jshell> var deque = new ArrayDeque<Integer>(List.of(2, 3, 4, 5));
deque ==> [2, 3, 4, 5]
jshell> var revDeque = deque.reversed();
revDeque ==> [5, 4, 3, 2]

jshell> revDeque.addFirst(6);
jshell> revDeque
revDeque ==> [6, 5, 4, 3, 2]
jshell> deque
deque ==> [2, 3, 4, 5, 6]

jshell> deque.removeLast();
$22 ==> 6
jshell> deque
deque ==> [2, 3, 4, 5]
jshell> revDeque
revDeque ==> [5, 4, 3, 2]
```

The `LinkedList<E>` Class

The `LinkedList` class overrides all sequence methods from the `List()` and the `Deque` interfaces. The `reversed()` method is covariantly overridden. Changes to the view returned by the `reversed()` method or

the underlying linked list always write through.

2.6 Sequenced Maps

A map contains *mappings* (also called *entries*) from a set of *key* objects to a set of *value* objects. A key can, at most, be associated with one value—that is, the key must be unique. **Figure 2.2** shows the type hierarchy under the root interface `Map` that represents all maps in the Java Collections Framework. The abstract classes that provide the basis on which concrete classes are implemented are not shown in **Figure 2.2**.

Figure 2.2 also shows how the `SequencedMap` interface and its subtypes that provide support for *sequenced maps* are retrofitted onto the inheritance hierarchy of maps. These sequenced interfaces and their sequenced subtypes (shown colored grey in **Figure 2.2**) impose an encounter order on the *entries* of a sequenced map and define operations on entries at either end. Maps are *not* iterable, and further, do *not* provide an iterator. Iteration over the entries in a map is normally done indirectly using an iterable keyset view of the map.

The core map interfaces and their concrete implementations are summarized in **Table 2.3**. Rows colored light grey in **Table 2.3** represent sequenced maps.

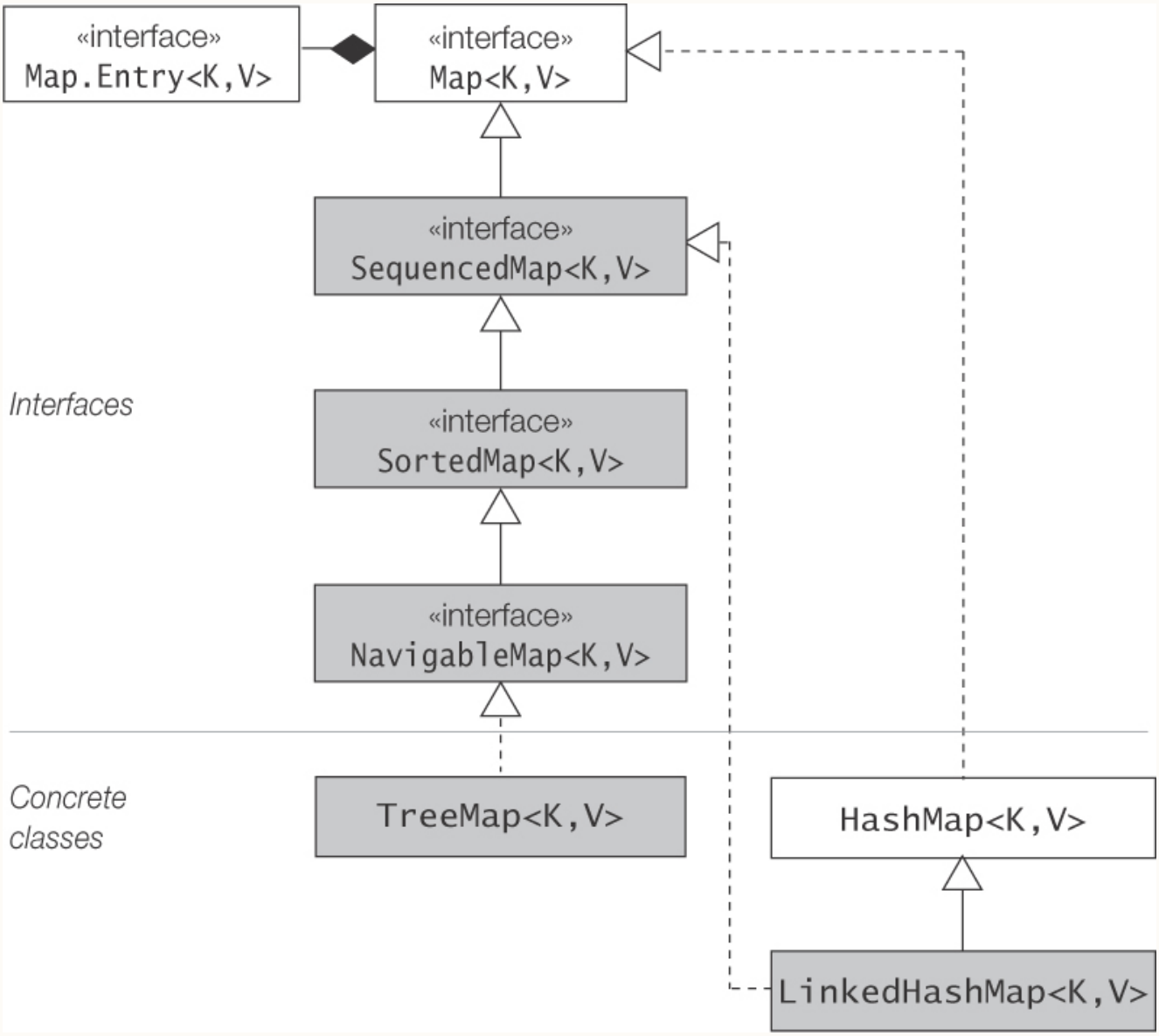


Figure 2.2 *Core Maps Inheritance Hierarchy*

Table 2.3 *Summary of Core Map Interfaces*

Interface	Description	Select Concrete Classes
`Map<K,V>`	A basic interface that defines operations for maintaining *mappings* of *keys* to *values*, where each key is distinct and can map to at most one value.	`HashMap<K,V>`, `LinkedHashMap<K,V>`, `TreeMap<K,V>`
`SequencedMap<K,V>` extends `Map<K,V>`	A sequenced map with a well-defined reversible encounter order on its mappings, and that allows operations at both ends.	`LinkedHashMap<K,V>`, `TreeMap<K,V>`
`SortedMap<K,V>` extends `SequencedMap<K,V>`	A sorted map that maintains its mappings sorted in *key order*.	`TreeMap<K,V>`
`NavigableMap<K,V>` extends `SortedMap<K,V>`	A navigable map is a sorted map extended with navigable methods to locate the closest match to a given key.	`TreeMap<K,V>`

The `SequencedMap<K, V>` Interface

A *sequenced map* is a map whose *entries* have a well-defined encounter order. Its behavior is dictated by the `SequencedMap` interface that defines methods to add, retrieve, and remove entries at either end of the map. In addition, it includes a method to create a *reverse-ordered view* of such a map, and methods to create three *sequenced views on its keyset, values collection, and entrySet*, respectively.

java.util.SequencedMap<K,V>

Click here to view code image

```
// Optional put methods (promoted from the NavigableMap<K, V> interface):
default V putFirst(K k, V v)
```

```
default V putLast(K k, V v)
```

The methods insert the specified mapping into the map if it is not present or they replace the value of the mapping if it is present. In either case, the specified mapping becomes the first *entry* or the last *entry* in this sequenced map's encounter order, respectively.

The methods return the value previously associated with `k`, or the `null` value.

The methods will throw an `UnsupportedOperationException` if the operation is not supported.

The default implementations of the `put*()` methods in the `SequencedMap` interface will throw an `UnsupportedOperationException.`

Click here to view code image

```
// Get methods (promoted from the NavigableMap<K, V> interface):
default Map.Entry<K, V> firstEntry()
default Map.Entry<K, V> lastEntry()
```

The methods return the first *entry* or the last *entry* currently in this sequenced map, respectively. The methods only retrieve, and do not remove the entry that is returned. Both methods return `null` if the map is empty.

The default implementations of these methods in the `SequencedMap` interface use an iterator on its entry set or entry set of its reverse-ordered view, respectively, to return *an unmodifiable copy* of the appropriate entry.

Click here to view code image

```
// Optional poll methods (promoted from the NavigableMap<K, V> interface):
default Map.Entry<K, V> pollFirstEntry()
default Map.Entry<K, V> pollLastEntry()
```

The methods *remove* and return the first *entry* or the last *entry* currently in this map, respectively.

The methods will throw an `UnsupportedOperationException` if the operation is not supported.

The default implementations of these methods in the `SequencedMap` interface use an iterator on its entry set or entry set of its reverse-ordered view, respectively, to return *an unmodifiable copy* of the appropriate entry.

Click here to view code image

```
// Create sequenced views on keys, values, and entries:
default SequencedSet<K>            sequencedKeySet()
```

```
default SequencedCollection<V>       sequencedValues()
default SequencedSet<Map.Entry<K, V>> sequencedEntrySet()
```

These methods return a *sequenced set view* of the map's *key set* (returned by `keySet()` method), *a sequenced collection view* of the map's *values collection* (returned by the `values()` method), and a *sequenced set view* of the map's *entry set* (returned by `entrySet()` method), respectively.

Click here to view code image

```
// Create the reverse-ordered view:
SequencedMap<K, V> reversed()
```

The method returns a *reverse-ordered* view of this map.

The `SequencedMap` interface does not provide an implementation for this method, and is thus declared as an `abstract` method.

Table 2.4 shows the behavior of the sequenced methods in concrete sequenced maps, as defined by the `SequencedMap` interface:

The `put*()` methods of the concrete class `TreeMap` will always throw an `UnsupportedOperationException` so that its defined encounter order (defined by *key sort order*) is not compromised.

The `firstEntry()`, `lastEntry()`, and `poll*Entry()` methods of both `TreeMap` and `LinkedHashMap` classes return a `null` value if the map is empty.

The `reversed()` method of the concrete sequenced maps returns a reverse-ordered view that is of the supertype, and methods for sequenced views on keys, values, and entries are all inherited from the respective superinterface.

Table 2.4 *Sequenced Methods in Select Concrete Sequenced Maps*

Maps	`putFirst()` `putLast()` (Optional)	`firstEntry()` `lastEntry()`	`pollFirstEntry()` `pollLastEntry()` (Optional)	`reversed()`	Sequenced Views on Keys, Values, and Entries
`TreeMap<K,V>` implements `NavigableMap<K,V>`	*overrides, throws* `UnsupportedOperationException`	*overrides, may return* `null`	*overrides, may return* `null`	*inherits, returns* `NavigableMap<K,V>`	*inherits*
`LinkedHashMap<K,V>` extends `HashMap<K,V>` implements `SequencedMap<K,V>`	*overrides*	*inherits, may return* `null`	*inherits, may return* `null`	*overrides, returns* `SequencedMap<K,V>`	*overrides*

Views on Keys, Values, and Entries of a `SequencedMap<K,V>`

The sequenced views on keys, values, and entries returned by the respective methods in sequenced maps behave as follows:

- The `add()` and the `addAll()` methods will throw `UnsupportedOperationException`, as these views are unmodifiable.
- The `reversed()` method on these sequenced views returns the reverse-ordered view created from the reverse-ordered view of the map itself. For example, the `reversed()` method called on a sequenced key set view of a sequenced map will return the sequenced key set view of the reverse-ordered sequenced map.
- Other methods on a sequenced view call the corresponding methods from the view that this sequenced view was created from. For example, for the `SequencedSet<K>` view returned by the `sequencedKeySet()` method, its other methods call the corresponding methods of the *key set view* of the map.

Composing Views

Views can also be created from sublists and submaps. In addition, views can be composed of other views. The following `jshell` session shows how a functionally equivalent reverse-ordered sequenced view of a map's key set can be created: either creating it from the sequenced key set of the map or creating a sequenced key set view of the reverse-ordered view of the map.

Click here to view code image

```
jshell> var tm = new TreeMap<Integer, String>();
tm ==> {}
jshell> tm.put(1, "One"); tm.put(2, "Two");
...
jshell> tm.put(3, "Three"); tm.put(4, "Four");
...
jshell> tm
tm ==> {1=One, 2=Two, 3=Three, 4=Four}
jshell> tm.sequencedKeySet().reversed();
$16 ==> [4, 3, 2, 1]
jshell> tm.reversed().sequencedKeySet();
$17 ==> [4, 3, 2, 1]
```

Static Entry Snapshots

The methods `firstEntry()`, `lastEntry()`, and `poll*Entry()` return a *static snapshot of an entry* (that is, a `Map.Entry<K, V>` instance) from the sequenced map at the time of the call. The value in this entry cannot be changed and written through to the underlying map using the `setValue()` method of the `Map.Entry` interface. The following `jshell` session illustrates the static nature of the entry returned by these methods:

Click here to view code image

```
jshell> tm
tm ==> {1=One, 2=Two, 3=Three, 4=Four}
jshell> var entry = tm.firstEntry();  // Static snapshot.
entry ==> 1=One
jshell> entry.setValue("Uno");
|  Exception java.lang.UnsupportedOperationException: not supported
|        at NullableKeyValueHolder.setValue (NullableKeyValueHolder.java:126)
|        at (#20:1)
```

The above behavior is in contrast to processing entries using an iterator on the key set of the map, where changes in the value of entries write through to the underlying map:

Click here to view code image

```
jshell> tm
tm ==> {1=One, 2=Two, 3=Three, 4=Four}
jshell> var nextEntry = tm.entrySet().iterator().next();
nextEntry ==> 1=One
jshell> nextEntry.setValue("Uno")
$41 ==> "One"
jshell> nextEntry
nextEntry ==> 1=Uno
jshell> tm
tm ==> {1=Uno, 2=Two, 3=Three, 4=Four}
```

The `SortedMap<K,V>` Interface

The `SortedMap` interface provides default implementation of the `put*()` methods that throw an `UnsupportedOperationException`, as *direct positioning* is not supported, since the position of an entry in the map is determined by the *sort order of its keys*.

The `firstEntry()`, `lastEntry()`, `poll*Entry()`, and the methods to create sequenced views on keys, values, and entries in the map, are inherited from its superinterface `SequencedMap`. The `reversed()` method is covariantly overridden to return a `SortedMap()`.

The `NavigableMap<K,V>` Interface

The `NavigableMap` interface inherits the `put*()` methods and the methods to create sequenced views on keys, values, and entries in the map from its superinterface `SortedMap`. The `firstEntry()`, `lastEntry()`, and `poll*Entry()` methods are declared as `abstract` methods. The `reversed()` method is covariantly overridden to return a `NavigableMap()`.

The `TreeMap<K,V>` Class

The `firstEntry()`, `lastEntry()`, and `poll*Entry()` methods of the `TreeMap` class are overridden from its superinterface `NavigableMap`. The `reversed()` method and the methods to create sequenced views on keys, values, and entries in the map are inherited from the superinterface `NavigableMap`. See also **Table 2.4, p. 59**.

Analogous to the `add*()` methods of the `TreeSet` class, the `put*()` methods of the `TreeMap` class also throw an `UnsupportedOperationException`, so as not to violate the key sort order of the entries:

Click here to view code image

```
jshell> var sortedMap = new TreeMap<Integer, String>();
sortedMap ==> {}
jshell> sortedMap.putFirst(50, "Java Actually");
|  Exception java.lang.UnsupportedOperationException
|        at TreeMap.putFirst (TreeMap.java:321)
|        at (#45:1)
```

The `LinkedHashMap<K,V>` Class

The `LinkedHashMap` interface inherits the `firstEntry()`, `lastEntry()`, and `poll*Entry()` methods from its superinterface `SequencedMap`. It overrides the `put*()` and `reversed()` methods, including those to create sequenced views on keys, values, and entries in the map from its superinterface `SequencedMap`. See also **Table 2.4, p. 59**.

Analogous to the `add*()` methods in the `LinkedHashSet` class, the `put*()` methods of the `LinkedHashMap` class add the entry at the appropriate end, unless it is a duplicate, in which case the existing entry is moved to the appropriate end. The `jshell` session below illustrates how a duplicate entry in a `LinkedHashMap` will be moved to the appropriate end. Note that the defined encounter order of the linked hash map in this example is insertion order.

Click here to view code image

```
// Repositioning the entry if present:
jshell> lhm
lhm ==> {1=One, 2=Two, 3=Three, 4=Four}
jshell> lhm.putFirst(3, "Three");
$50 ==> "Three"
jshell> lhm
lhm ==> {3=Three, 1=One, 2=Two, 4=Four}
```

However, if the value of an *existing* entry is changed, it will be repositioned at the appropriate end, as can be seen if we continue the `jshell` session from above:

Click here to view code image

```
// Changing exiting entry, repositions the entry:
jshell> lhm
lhm ==> {3=Three, 1=One, 2=Two, 4=Four}
jshell> lhm.putFirst(2, "II");
$52 ==> "Two"
jshell> lhm
lhm ==> {2=II, 3=Three, 1=One, 4=Four}
```

2.7 Unmodifiable Sequenced Collections, Sets, and Maps

Analogous to methods for creating *unmodifiable views* on collections and maps, the `java.util.Collections` utility class also provides static methods to create *unmodifiable sequenced views* that are backed by *sequenced* collections, sets, and maps. These views are useful when an unmodifiable copy of a sequence collection, set or map is desired.

java.util.Collections

Click here to view code image

```
<E> SequencedCollection<E>
      unmodifiableSequencedCollection(SequencedCollection<? extends E> c)
<E> SequencedSet<E>
      unmodifiableSequencedSet(SequencedSet<? extends E> set)
<K, V> SequencedMap<K, V>
      unmodifiableSequencedMap(SequencedMap<? extends K, ? extends V> map)
```

Return an *unmodifiable sequenced view* of the sequenced collection, set or map passed as an argument, respectively.

Query operations on the returned sequenced collection, set or map are delegated to the *underlying sequenced collection*, *set* or *map*, and changes in the underlying sequenced collection, set or map are reflected in the unmodifiable sequenced view.

Any attempt to modify the unmodifiable view will result in an `UnsupportedOperationException`.

The code below creates an *unmodifiable sequenced view* of a sequenced map whose values are mutable.

Click here to view code image

```
// Mutable courses:
StringBuilder mc1 = new StringBuilder("Java I");
StringBuilder mc2 = new StringBuilder("Java II");
StringBuilder mc3 = new StringBuilder("Java III");

// Sequenced map:
SequencedMap<Integer, StringBuilder> underlyingSeqMap = new TreeMap<>();
underlyingSeqMap.put(100, mc1); underlyingSeqMap.put(200, mc2);
underlyingSeqMap.put(300, mc3);
System.out.println("Sequenced map:       " + underlyingSeqMap);
//    Sequenced map:      {100=Java I, 200=Java II, 300=Java III}

// Unmodifiable sequenced view of a sequenced map:
SequencedMap<Integer, StringBuilder> unmodSeqMapView
          = Collections.unmodifiableSequencedMap(underlyingSeqMap);
System.out.println("Unmodifiable sequenced view: " + unmodSeqMapView);
//    Unmodifiable sequenced view: {100=Java I, 200=Java II, 300=Java III}
```

As in the case of unmodifiable collections, the code below throws an `UnsupportedOperationException` when an attempt is made to structurally change the unmodifiable sequenced view of a sequenced map.

Click here to view code image

```
// UnsupportedOperationException at (1), (2), and (3):
unmodSeqMapView.put(400, new StringBuilder("Java Now"));        // (1)
unmodSeqMapView.remove(300);                                    // (2)
unmodSeqMapView.replace(200, new StringBuilder("First Java"));  // (3)
```

However, changes to the underlying sequenced map are reflected in the unmodifiable sequenced view:

Click here to view code image

```
underlyingSeqMap.remove(200);
System.out.println("Underlying sequenced map:    " + underlyingSeqMap);
System.out.println("Unmodifiable sequenced view: " + unmodSeqMapView);
//    Underlying sequenced map:    {100=Java I, 300=Java III}
//    Unmodifiable sequenced view: {100=Java I, 300=Java III}
```

Since the values in the unmodifiable sequenced view of the sequenced map are mutable, the code below shows that changing a value of an entry in the unmodifiable sequenced view is reflected in the underlying sequenced map.

Click here to view code image

```
StringBuilder mutableCourse = unmodSeqMapView.get(300);// Get the course.
mutableCourse.replace(5, 8, "Complete");            // Change the course name.
System.out.println("Underlying sequenced map:    " + underlyingSeqMap);
System.out.println("Unmodifiable sequenced view: " + unmodSeqMapView);
//  Underlying sequenced map:    {100=Java I, 300=Java Complete}
//  Unmodifiable sequenced view: {100=Java I, 300=Java Complete}
```

2.8 Sequenced Concurrent Collections

We assume familiarity with the core concurrent interfaces and classes in the `java.util.concurrent` package, prior to Java 21.

Majority of the collections in the `java.util` package are not thread-safe. The concurrent collections in `java.util.concurrent` package that provide thread-safety have also been retrofitted to comply with the requirements of sequenced collections introduced in the `java.util` package (shown in grey in Figure 2.3).

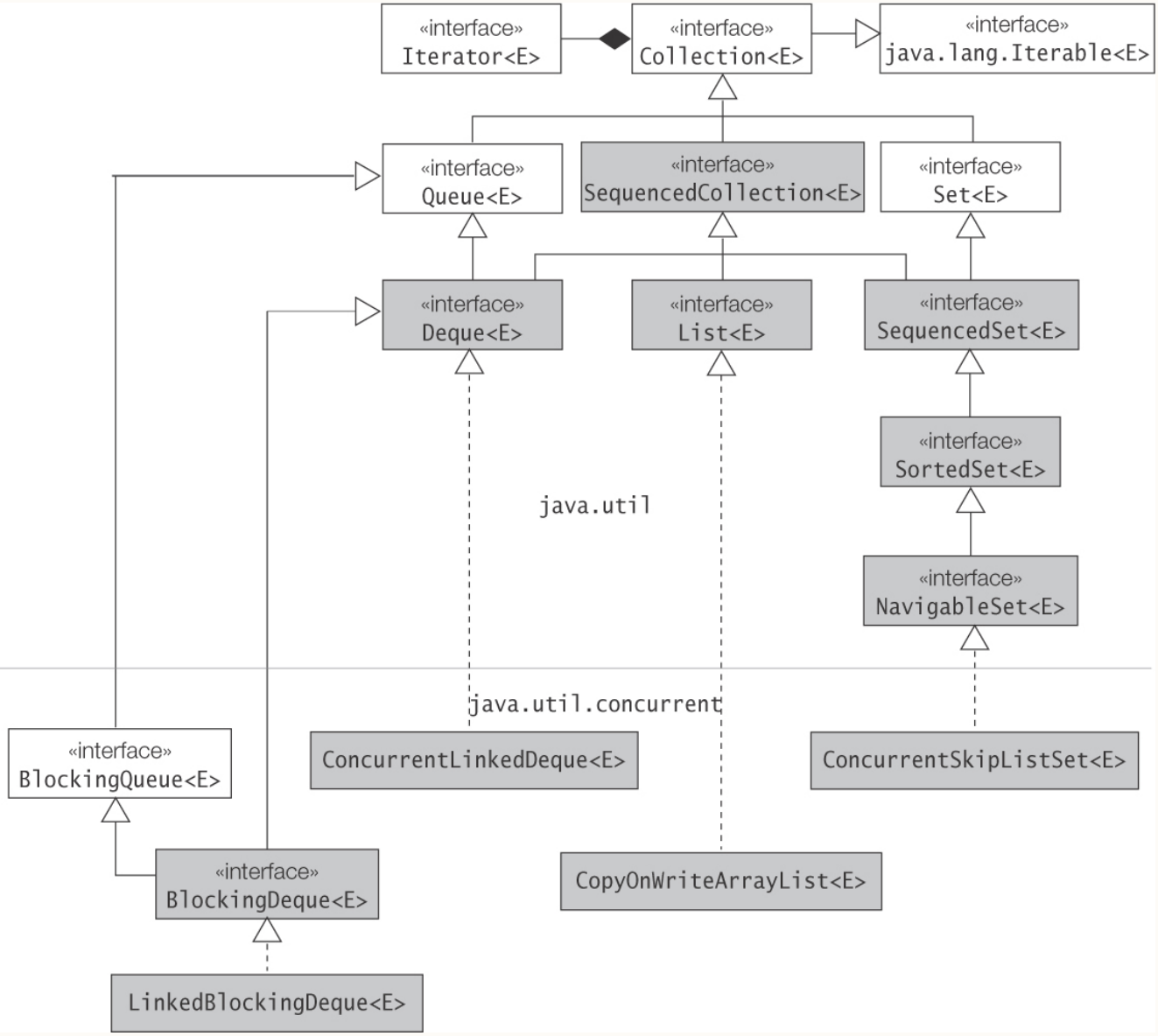


Figure 2.3 *Core Concurrent Collections Inheritance Hierarchy*

The concurrent collections provided by the `java.util.concurrent` package use special-purpose locking mechanisms to enable multiple threads to operate concurrently, without explicit locking on the collec-

tion and with minimal contention. **Table 2.5** gives a brief description of select concurrent collection classes and interfaces found in the `java.util.concurrent` package. Rows colored light grey in **Table 2.5** represent sequenced concurrent interfaces and classes.

Table 2.5 *Concurrent Collections in the* `java.util.concurrent` *Package*

Concurrent Collections	Description
`ConcurrentSkipListSet<E>` implements `NavigableSet<E>`	An unbounded, non-blocking, concurrent `NavigableSet` whose elements are sorted according to the natural ordering or by a comparator that is provided in the constructor.
`ConcurrentLinkedDeque<E>` implements `Deque<E>`	An unbounded, non-blocking, concurrent deque based on linked nodes.
interface `BlockingQueue<E>` extends `Queue<E>`	A `Queue` that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element.
interface `BlockingDeque<E>` extends `BlockingQueue<E>`, `Deque<E>`	A `Deque` that additionally supports blocking operations that wait for the deque to become non-empty when retrieving an element, and wait for space to become available in the deque when storing an element.
`LinkedBlockingDeque<E>` implements `BlockingDeque<E>`	An optionally bounded blocking deque based on doubly linked nodes.

Concurrent Collections	Description
CopyOnWriteArrayList<E> implements List<E>	A thread-safe variant of ArrayList in which all mutative operations (add, set, etc.) are implemented by making a fresh copy of the underlying array.

The sequenced concurrent collections in the java.util.concurrent package have the SequencedCollection interface as their common supertype, analogous to the sequenced collections in the java.util package (**Figure 2.3**). They have a defined encounter order and provide uniform operations at both ends, including a reverse-ordered view. **Table 2.6** summarizes the behavior of the sequenced methods in select sequenced concurrent collections, as defined by the SequencedCollection interface (**p. 49**).

It should not come as a surprise that the add*() methods of a ConcurrentSkipListSet will throw an UnsupportedOperationException, as the elements of this sequenced concurrent collection have a sort order which would be violated if this operation were allowed. The add*() methods of a ConcurrentLinkedDeque and LinkedBlockingDeque throw a NullPointerException if the element is null. A LinkedBlockingDeque can also throw an IllegalStateException if the deque is full.

The get*() and remove*() methods throw a NoSuchElementException if the sequenced concurrent collection is empty. The remove*() methods of a ConcurrentSkipListSet can also throw an UnsupportedOperationException.

The inherited reversed() method of a concrete sequenced concurrent collection in **Table 2.6** returns a reverse-ordered collection whose type is that of its superinter-face. Analogous to the thread-unsafe sequenced collections, changes in the reverse-ordered view write through to the underlying sequenced concurrent collection, and vice versa.

Table 2.6 *Sequenced Methods in Concrete Sequenced Concurrent Collections*

Concurrent Sequenced Collections	**addFirst()** **addLast()** **(Optional)**	**getFirst()** **getLast()**	**removeFirst()** **removeLast()** (Optional)	**reversed()**
ConcurrentSkipListSet<E> implements NavigableSet<E>	*overrides, throws* UnsupportedOperationException	*inherits, may throw* NoSuchElementException	*inherits, may throw* NoSuchElementException, *may throw* UnsupportedOperationException	*inherits, returns* NavigableSet<E>

Concurrent Sequenced Collections	`addFirst()` `addLast()` (Optional)	`getFirst()` `getLast()`	`removeFirst()` `removeLast()` (Optional)	`reversed()`
ConcurrentLinkedDeque<E> implements Deque<E>	*overrides, may throw* NullPointer-Exception	*overrides, may throw* NoSuchElement-Exception	*overrides, may throw* NoSuchElement-Exception	*inherits, returns* Deque<E>
LinkedBlockingDeque<E> implements BlockingDeque<E>	*overrides, may throw* NullPointer-Exception, *may throw* IllegalState-Exception	*overrides, may throw* NoSuchElement-Exception	*overrides, may throw* NoSuchElement-Exception	*inherits, returns* Deque<E>
CopyOnWriteArrayList<E> implements List<E>	*overrides*	*overrides, may throw* NoSuchElement-Exception	*overrides, may throw* NoSuchElement-Exception	*overrides, returns* List<E>

2.9 Sequenced Concurrent Maps

Figure 2.4 shows how the `SequencedMap` interface and its subtypes that provide support for *sequenced concurrent maps* are retrofitted onto the inheritance hierarchy of sequenced maps. The sequenced concurrent maps in the `java.util.concurrent` package have the `SequencedMap` interface as their common supertype, analogous to the sequenced maps in the `java.util` package. The sequenced interfaces and their sequenced subtypes (shown colored grey in Figure 2.4) impose an encounter order on the *entries* of a sequenced map and define operations on entries at either end, including a reverse-ordered view.

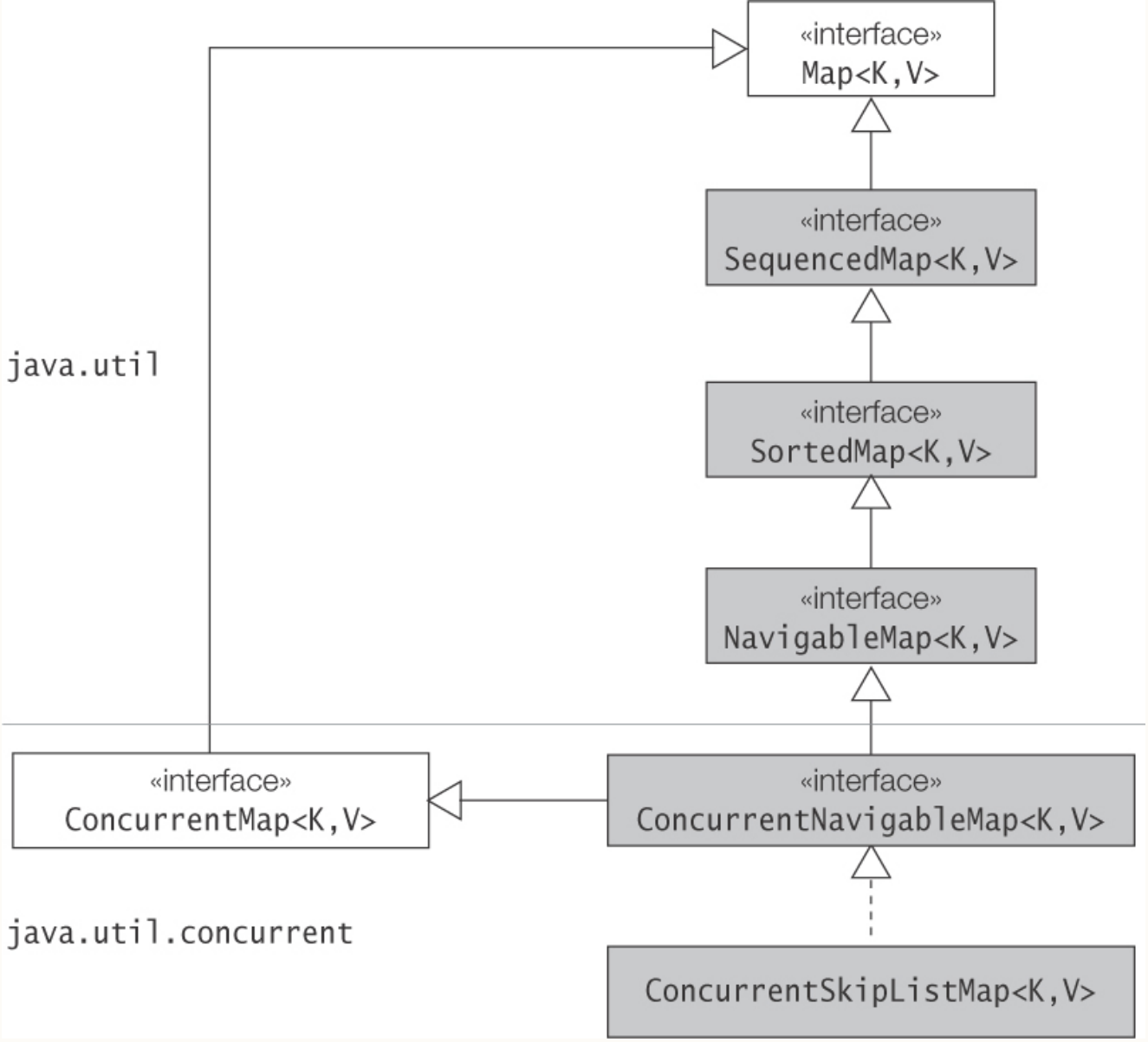


Figure 2.4 *Core Concurrent Maps Inheritance Hierarchy*

The core concurrent map interfaces and their concrete implementations are summarized in **Table 2.7**. Rows colored light grey in **Table 2.7** represent sequenced concurrent maps. **Table 2.8** summarizes the behavior of the sequenced methods for the `ConcurrentSkipListMap` class, as defined by the `SequencedMap` interface (**p. 58**).

As sequenced maps, the class `ConcurrentSkipListMap` in the `java.util.concurrent` package exhibits analogous behavior to the thread-unsafe class `TreeMap` in the `java.util` package:

- Behavior of the `put*()` methods that throw an `UnsupportedOperationException` (**p. 59**).
- Behavior of views on keys, values, and entries (**p. 60**).
- Composing views (**p. 60**).
- Immutability of static entry snapshots (**p. 60**).

Table 2.7 *Concurrent Maps in the* `java.util.concurrent` *Package*

Concurrent Maps	Description
`interface ConcurrentMap<K,V> extends Map<K,V>`	A `Map` providing thread-safety and atomicity guarantees
Click here to view code image `interface ConcurrentNavigableMap<K,V> extends ConcurrentMap<K,V>, NavigableMap<K,V>`	A `ConcurrentMap<K, V>` supporting `NavigableMap<K,V>` operations, and recursively so for its navigable sub-maps
`ConcurrentSkipListMap<K,V> implements ConcurrentNavigableMap<K,V>`	A scalable concurrent `ConcurrentNavigableMap<K,V>` implementation that is highly efficient for traversal

Table 2.8 *Sequenced Methods in the* `ConcurrentSkipListMap<K,V>` *class*

Concurrent Sequenced Map	`putFirst()` `putLast()` (Optional)	`firstEntry()` `lastEntry()`	`pollFirstEntry()` `pollLastEntry()` (Optional)	`reversed()`	Sequenced Views on Keys, Values, and Entries
`ConcurrentSkipListMap<K,V>` implements `ConcurrentNavigableMap<K,V>`	*overrides, throws* `UnsupportedOperationException`	*overrides, may return* `null`	*overrides, may return* `null`	*inherits, returns* `NavigableMap<K,V>`	*inherits*



Review Questions

2.1 Given the following code:

Click here to view code image

```
SequencedSet<Integer> values = new LinkedHashSet<>();
values.addFirst(Integer.valueOf(1));
values.addFirst(Integer.valueOf(2));
values.addLast(Integer.valueOf(3));
values.addLast(Integer.valueOf(2));
values.add(Integer.valueOf(3));
System.out.println(values);
```

What is the result?

1. `[1, 2, 3]`
2. `[2, 1, 3]`
3. `[3, 2, 1]`
4. `[1, 3, 2]`
5. `[2, 1, 3, 2]`
6. `[2, 1, 3, 1]`

2.2 Which one of the code fragments prints `[A, B, C]`?

Select the two correct answers.

1. **Click here to view code image**

```
SequencedCollection<String> letters = new TreeSet<>();
letters.addFirst("C");
letters.addFirst("B");
letters.addFirst("A");
System.out.println(letters);
```

2. **Click here to view code image**

```
SequencedCollection<String> letters = new HashSet<>();
letters.addLast("A");
letters.addLast("B");
letters.addLast("C");
System.out.println(letters);
```

3. **Click here to view code image**

```
SequencedCollection<String> letters = new LinkedHashSet<>();
letters.addFirst("C");
letters.addFirst("B");
letters.addFirst("A");
System.out.println(letters);
```

4. **Click here to view code image**

```
SequencedSet<String> letters = new HashSet<>();
letters.addFirst("C");
letters.addFirst("B");
letters.addFirst("A");
System.out.println(letters);
```

5. Click here to view code image

```
SequencedCollection<String> letters = new TreeSet<>();
letters.add("C");
letters.add("B");
letters.add("A");
System.out.println(letters);
```

2.3 Which one of the following code fragments prints `true`?

1. Click here to view code image

```
SequencedMap<String,String> map = new TreeMap<>();
map.put("A","C");
map.put("B","B");
map.put("C","A");
SequencedSet<String> keys = map.sequencedKeySet().reversed();
map.reversed();
SequencedCollection<String> values = map.sequencedValues();
System.out.println(keys.getFirst().equals(values.getFirst()));
```

2. Click here to view code image

```
SequencedMap<String,String> map = new TreeMap<>();
map.put("A","C");
map.put("B","B");
map.put("C","A");
SequencedSet<String> keys = map.sequencedKeySet().reversed();
map = map.reversed();
SequencedCollection<String> values = map.sequencedValues();
System.out.println(keys.getFirst().equals(values.getFirst()));
```

3. Click here to view code image

```
SequencedMap<String,String> map = new TreeMap<>();
map.put("A","C");
map.put("B","B");
map.put("C","A");
SequencedSet<String> keys = map.sequencedKeySet();
SequencedCollection<String> values = map.sequencedValues();
System.out.println(keys.getFirst().equals(values.getFirst()));
```

4. Click here to view code image

```
SequencedMap<String,String> map = new TreeMap<>();
map.put("A","C");
map.put("B","B");
map.put("C","A");
SequencedSet<String> keys = map.sequencedKeySet().reversed();
SequencedCollection<String> values = map.sequencedValues().reversed();
System.out.println(keys.getFirst().equals(values.getFirst()));
```

2.4 Given the following code:

Click here to view code image

```
public class Q4 {
  public static void main(String[] args) {
    SequencedMap<Integer, String> lhm = new LinkedHashMap<>();
    String[] numbers = { "Zero", "One", "Two", "Three", "Four" };
    for (int i = 0; i < 5; i++) {
      if (i % 2 != 0) {
        lhm.putFirst(i, numbers[i]);
      } else {
        lhm.putLast(i, numbers[i]);
      }
      if (i == 2) {
        lhm.putFirst(i, numbers[i]);
      }
    }
    System.out.println(lhm);
  }
}
```

What is the result of compiling and running the program?

1. {3=Three, 1=One, 0=Zero, 2=Two, 4=Four}
2. {3=Three, 2=Two, 1=One, 0=Zero, 4=Four}
3. {4=Four, 2=Two, 1=One, 0=Zero, 3=Three}
4. {4=Four, 1=One, 0=Zero, 2=Two, 3=Three}

Previous chapter

< 1. Pattern Matching for Java Objects

Next chapter

3. Virtual Threads >